

QUESTIONS

Domanda 1

Implementare la funzione `forgettable_channel<T>()` che crea un canale multi-produttore/ricevitore-singolo le cui semantiche estendono quelle di `std::sync::mpsc::channel` con la possibilità di annullare un messaggio già accodato prima che venga elaborato dal ricevitore.

Interfacce fornite

I tratti che seguono definiscono il contratto pubblico del canale; non devono essere modificati.

```
1 // Forgettable
2 // Handle restituito da 'send()'. Il metodo 'forget()' tenta di annullare
3 // il messaggio corrispondente ancora in coda:
4 // - restituisce 'true' se il messaggio era ancora in attesa o il canale
5 //   è stato chiuso prima che il messaggio fosse consegnato al ricevitore.
6 // - restituisce 'false' se il ricevitore lo aveva già elaborato.
7
8 // ForgettableSender<T>: Clone
9 // Lato mittente. 'send(t)' restituisce 'Some(handle)' se il messaggio è
10 // stato accodato con successo, 'None' se il ricevitore non esiste più.
11
12 // ForgettableReceiver<T>
13 // Lato ricevitore. 'recv()' blocca finché non è disponibile un messaggio
14 // non annullato e restituisce 'None' solo quando il canale è chiuso e la
15 // coda è vuota. I messaggi annullati vengono scartati silenziosamente.
```

Suggerimenti implementativi

1. Creare la coppia mittente/ricevitore sottostante (ad esempio tramite `std::sync::mpsc::channel`).
2. Avvolgere il mittente in un tipo concreto che implementi sia `ForgettableSender<T>` sia `Clone`, così da supportare più produttori.
3. Avvolgere il ricevitore in un tipo concreto che implementi `ForgettableReceiver<T>`.

Test di accettazione

Tutti i test devono passare senza modifiche.

SOLUTIONS

Rust Implementation

Di seguito un'implementazione idiomatica in Rust che rispetta le specifiche fornite, utilizzando `std::sync::mpsc::channel` internamente. Per tenere traccia dello stato di annullamento di ciascun messaggio inviato, il mittente avvolge ogni messaggio `T` e un flag atomico `Arc<AtomicBool>` condiviso con l'handle restituito.

```
1 use std::sync::{mpsc, Arc};
2 use std::sync::atomic::{AtomicBool, Ordering};
3
4 pub trait Forgettable {
5     fn forget(&self) -> bool;
6 }
7
8 pub trait ForgettableSender<T>: Clone {
9     fn send(&self, t: T) -> Option<Box<dyn Forgettable>>;
10 }
11
12 pub trait ForgettableReceiver<T> {
13     fn recv(&self) -> Option<T>;
14 }
15
16 // =====
17 // Implementation
18 // =====
19
20 pub struct Handle {
21     cancelled: Arc<AtomicBool>,
22     received: Arc<AtomicBool>,
23 }
24
25 impl Forgettable for Handle {
26     fn forget(&self) -> bool {
27         if self.received.load(Ordering::Acquire) {
28             false
29         } else {
30             self.cancelled.store(true, Ordering::Release);
31             true
32         }
33     }
34 }
35
36 // Wrapper for the message containing the flags
37 struct MessageWrapper<T> {
38     item: T,
39     cancelled: Arc<AtomicBool>,
40     received: Arc<AtomicBool>,
41 }
42
43 pub struct SenderImpl<T> {
44     tx: mpsc::Sender<MessageWrapper<T>>,
45 }
46
47 impl<T> Clone for SenderImpl<T> {
48     fn clone(&self) -> Self {
49         SenderImpl { tx: self.tx.clone() }
50     }
51 }
52
53 impl<T: 'static> ForgettableSender<T> for SenderImpl<T> {
54     fn send(&self, t: T) -> Option<Box<dyn Forgettable>> {
55         let cancelled = Arc::new(AtomicBool::new(false));
56         let received = Arc::new(AtomicBool::new(false));
57
58         let handle = Handle {
59             cancelled: Arc::clone(&cancelled),
60             received: Arc::clone(&received),
61         };
62
63         let msg = MessageWrapper {
64             item: t,
65             cancelled,
```

```

66         received,
67     };
68
69     match self.tx.send(msg) {
70         Ok(_) => Some(Box::new(handle)),
71         Err(_) => None, // Receiver disconnected
72     }
73 }
74 }
75
76 pub struct ReceiverImpl<T> {
77     rx: mpsc::Receiver<MessageWrapper<T>>,
78 }
79
80 impl<T> ForgettableReceiver<T> for ReceiverImpl<T> {
81     fn recv(&self) -> Option<T> {
82         loop {
83             match self.rx.recv() {
84                 Ok(msg) => {
85                     msg.received.store(true, Ordering::Release);
86                     if !msg.cancelled.load(Ordering::Acquire) {
87                         return Some(msg.item);
88                     }
89                     // If cancelled, loop and try the next one
90                 }
91                 Err(_) => return None, // Channel closed and empty
92             }
93         }
94     }
95 }
96
97 pub fn forgettable_channel<T: 'static>() -> (impl ForgettableSender<T>, impl
98     ForgettableReceiver<T>) {
99     let (tx, rx) = mpsc::channel();
100     (SenderImpl { tx }, ReceiverImpl { rx })

```